

IPC - Message Queue

X/Open System Interface (XSI) - IPCs

- *Message Queues, Shared Memory, and Semaphore* are XSI-IPCs.
- X/Open group for Unix Systems – was a consortium founded by several European UNIX system manufacturers in 1984 to identify and promote open standards in the field of information technology, later incorporated in 1987 as X/Open Company.
- The original aim was to define a single specification for operating systems derived from UNIX, to increase the interoperability of applications and reduce the cost of porting software.
- Its original members were Bull, ICL, Siemens, Olivetti, and Nixdorf – a group sometimes referred to as BISON. Philips and Ericsson joined in 1985.

Message Queue (MQ)

- Two or more processes can access the common memory, MQ, to communicate.
- The sending process places a message onto a queue (via some message-passing module) which can be read by another process.
- Processes must share a common key in order to gain access to the queue.
- A MQ is a one-way pipe: one process writes to the queue, and another reads the data in the order it was written until an end-of-data condition occurs.
- Once a message is read it is deleted from the queue by the kernel.

Message Queue (Cont')

- When the queue is created, the message size (bytes per message, usually fairly small) and queue length (maximum number of pending messages) are set.
- A MQ is a linked list of messages stored within the kernel and identified by a MQ ID (or type).
- Each read and write creates a system call to the kernel.
- The MQ helps to eliminate the occurrences of race conditions but comes at the expense of performance due to the system call interrupt.
- Ex.: ps1 writes msg A, D, F,
ps2 reads A, ps3 reads D, ps4 reads F,
none of ps2, ps3, ps4 knows all the data.

ftok() and ipc_perm Structure for XSI-IPC

```
#include <sys/ipc.h>
// return a key on success, -1 on error
key_t ftok(const char *path, int id);
```

- The key is used in the subsequent calls to `msgget()`, `msgsnd()`, `msgrcv()`, `semget()`, and `shmget()`.
- The path argument is the pathname of an existing file.
- Only lower 8 bit of id are used when generating a queue.
- XSI IPC is associated with `ipc_perm` structure.

```
struct ipc_perm {
    uid_t uid; /* owner's effective user ID */
    gid_t gid; /* owner's effective group ID */
    uid_t cuid; /* creator effective user ID */
    gid_t cgid; /* creator effective group ID */
    mode_t mode /* access mode */
};
```

Message Queue Structure

Each queue has the `msqid_ds` structure associated with it.

```
struct msqid_ds {
    struct ipc_perm msg_perm;
    struct msg *msg_first; /* first message on queue */
    struct msg *msg_last; /* last message in queue */
    time_t msg_stime; /* last msgsnd time */
    time_t msg_rtime; /* last msgrcv time */
    time_t msg_ctime; /* last change time */
    /* wwait and rwait are pointers into the kernel's wait queue.
       They are used when a queue is full and the process
       is waiting for an opening. */
    struct wait_queue *wwait;
    struct wait_queue *rwait;
    ushort msg_cbytes; /* Total number of bytes residing on the queue */
    ushort msg_qnum; /* number of message on queue */
    ushort msg_qbytes; /* max number of bytes on queue */
    ushort msg_lspid; /* pid of last msgsnd */
    ushort msg_lrpid; /* last receive pid */
};
```

Message Queue Structure (Cont')

```
struct msg {  
    struct msg *msg_next;    //next message on queue  
    long  msg_type;  
    char *msg_spot;    //A pointer to the beginning of the message.  
    short msg_ts;    // message text size  
};
```

If a new queue is created, the `msqid_ds` structure are initiated.

- `msg_qnum`, `msg_lspid`, `msg_lrpid`, `msg_stime`, and `msg_rtime` are all set to 0.
- `msg_ctime` is set to the current time
- `msg_qbyte` is set to the system limit.

MQ APIs: `msgget()`, `msgsnd()`, `msgrcv()` and `msgctl()`

- A new message queue is created or opened by `msgget()`.
- A new messages are added to the end of a queue by `msgsnd()`.
- Messages are fetched from a queue by `msgrcv()`.
- We don't have to fetch the message in a FIFO order. Instead, we can fetch messages based on their type field.
- The `msgctl()` system call provides a variety of message control operations.

msgget() and msgctl()

```
#include <sys/msg.h>
/* returns the mq id for a newly created mq, or an existing
   mq id with the same key value; -1 on error */
int msgget (key_t key, int msgflg);
```

msgflg values:

- **IPC_CREAT**: Create the queue if it doesn't already exist in the kernel.
- **IPC_EXCL**: When used with **IPC_CREAT**, fail if queue already exists.

```
// Return 0 on success, -1 error
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

The `msgctl()` provides message control operations as specified by `cmd`.

- **IPC_STAT** retrieves the `msqid_ds` structure for a queue, and stores it in the address of the `buf` argument.
- **IPC_SET** sets the value of the `ipc_perm` member of the `msqid_ds` structure for a queue. Takes the values from the `buf` argument.
- **IPC_RMID** removes/destroys the queue from the kernel .

msgsnd()

```
#include <sys/msg.h>
// Return 0 on success, -1 error
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
```

The `msgsnd()` sends a message to the queue associated with message queue identifier `msqid`. On success, it results:

- `msg_qnum` is incremented by 1.
- `msg_lspid` is set to the calling process' pid.
- `msg_stime` is set to the current time.
- The `msgflg` argument can be set to 0 (ignored).

The argument `msgp` must point to a user-defined buffer that must first contain a field of type `long int` that specifies the type of the message, and then a data portion that holds the data of the message. Ex.

```
struct mymsg {
    long int mtype; /* positive message type */
    char mtext[n]; /* message data of n bytes */
}
```

msgrcv()

```
#include <sys/msg.h>
/* returns the number of bytes copied into message buffer; -1 on error */
int msgrcv(int msqid, void *msgp, int msgsz, long msgtyp, int msgflg);
```

The `msgrcv()` reads a message from the queue associated with `msqid` and places it in the user-defined structure that `msgp` points to. On success, it results:

- `msg_qnum` is decremented by 1.
- `msg_lrp` is set to the calling process' pid.
- `msg_rtime` is set to the current time.

The `msgtyp` Specifies the type of message requested as follows:

- If `msgtyp` is 0, the first message on the queue is received.
- If `msgtyp > 0`, the first message of type equal to `msgtyp` is received.
- If `msgtyp < 0`, the first message of the lowest type that is less than or equal to the absolute value of `msgtyp` is received.

`msgflg` specifies the action to be taken if a message of the desired type is not in the queue.

Code Example – Sender

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct my_msgbuf { /* user message type with 200 byte per message */
    long mtype;
    char mtext[200];
};
int main(void) {
    struct my_msgbuf buf;
    int msqid;
    key_t key;
    if ((key = ftok("sender.c", 'B')) == -1) { //create a key
        perror("ftok error"); exit(1);
    }
    if ((msqid = msgget(key, 0644 | IPC_CREAT)) == -1) { /*create a message queue */
        perror("msgget error"); exit(1);
    }
    printf("Enter lines of text, ^D to quit:\n");
    buf.mtype = 1; /* we don't really care in this case, just used as FIFO*/
    while (fgets(buf.mtext, sizeof(buf.mtext), stdin) != NULL) {
        if (msgsnd(msqid, (struct msgbuf *)&buf, sizeof(buf), 0) == -1)
            perror("msgsnd error");
    }
    if (msgctl(msqid, IPC_RMID, NULL) == -1) { /* remove message queue by calling msgctl */
        perror("msgctl error");
        exit(1);
    }
    exit(0);
}
```

Code Example – Receiver

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct my_msgbuf {
    long mtype;
    char mtext[200];
};
int main(void) {
    struct my_msgbuf buf;
    int msqid;
    key_t key;
    if ((key = ftok("sender.c", 'B')) == -1) { //create a key
        perror("ftok error");    exit(1);
    }
    if ((msqid = msgget(key, 0644)) == -1) { /* open message queue */
        perror("msgget error");    exit(1);
    }
    printf("ready to receive messages. \n");
    for(;;) {
        /* get each message from the message queue */
        if (msgrcv(msqid, (struct msgbuf *)&buf, sizeof(buf), 0, 0) == -1) {
            perror("msgrcv error");
            exit(1);
        }
        printf("receiver: \"%s\"\n", buf.mtext);
    }
    return 0;
}
```